Chapter 11. Conditional Logic

In certain situations, you may want your SQL logic to branch in one direction or another depending on the values of certain columns or expressions. This chapter focuses on how to write statements that can behave differently depending on the data encountered during statement execution. The mechanism used for conditional logic in SQL statements is the `case` expression, which can be utilized in `select`, `insert`, `update`, and `delete` statements.

What Is Conditional Logic?

Conditional logic is simply the ability to take one of several paths during program execution. For example, when querying customer information, you might want to include the `customer.active` column, which stores 1 to indicate active and 0 to indicate inactive. If the query results are being used to generate a report, you may want to translate the value to improve readability. While every database includes built-in functions for these types of situations, there are no standards, so you would need to remember which functions are used by which database. Fortunately, every database's SQL implementation includes the `case` expression, which is useful in many situations, including simple translations:

```
mysql> SELECT first_name, last_name,
    ->   CASE
    ->     WHEN active = 1 THEN 'ACTIVE'
    ->     ELSE 'INACTIVE'
    ->   END activity_type
    -> FROM customer;
+-------------+--------------+---------------+
| first_name  | last_name    | activity_type |
+-------------+--------------+---------------+
| MARY        | SMITH        | ACTIVE        |
| PATRICIA    | JOHNSON      | ACTIVE        |
| LINDA       | WILLIAMS     | ACTIVE        |
| BARBARA     | JONES        | ACTIVE        |
| ELIZABETH   | BROWN        | ACTIVE        |
| JENNIFER    | DAVIS        | ACTIVE        |
```

```
...
| KENT       | ARSENAULT    | ACTIVE       |
| TERRANCE   | ROUSH        | INACTIVE     |
| RENE       | MCALISTER    | ACTIVE       |
| EDUARDO    | HIATT        | ACTIVE       |
| TERRENCE   | GUNDERSON    | ACTIVE       |
| ENRIQUE    | FORSYTHE     | ACTIVE       |
| FREDDIE    | DUGGAN       | ACTIVE       |
| WADE       | DELVALLE     | ACTIVE       |
| AUSTIN     | CINTRON      | ACTIVE       |
+------------+--------------+--------------+
599 rows in set (0.00 sec)
```

This query includes a `case` expression to generate a value for the `activity_type` column, which returns the string "ACTIVE" or "INACTIVE" depending on the value of the `customer.active` column.

The case Expression

All of the major database servers include built-in functions designed to mimic the if-then-else statement found in most programming languages (examples include Oracle's `decode()` function, MySQL's `if()` function, and SQL Server's `coalesce()` function). `case` expressions are also designed to facilitate if-then-else logic but enjoy two advantages over built-in functions:

- The `case` expression is part of the SQL standard (SQL92 release) and has been implemented by Oracle Database, SQL Server, MySQL, PostgreSQL, IBM UDB, and others.
- `case` expressions are built into the SQL grammar and can be included in `select`, `insert`, `update`, and `delete` statements.

The next two subsections introduce the two different types of `case` expressions. This is followed by some examples of `case` expressions in action.

Searched case Expressions

The `case` expression demonstrated earlier in the chapter is an example of a *searched* `case` *expression*, which has the following syntax:

```
CASE
  WHEN C1 THEN E1
  WHEN C2 THEN E2
  ...
  WHEN CN THEN EN
  [ELSE ED]
END
```

In the previous definition, the symbols `C1`, `C2`, ..., `CN` represent conditions, and the symbols `E1`, `E2`, ..., `EN` represent expressions to be returned by the `case` expression. If the condition in a `when` clause evaluates to `true`, then the `case` expression returns the corresponding expression. Additionally, the `ED` symbol represents the default expression, which the `case` expression returns if *none* of the conditions `C1`, `C2`, ..., `CN` evaluate to `true` (the `else` clause is optional, which is why it is enclosed in square brackets). All the expressions returned by the various `when` clauses must evaluate to the same type (e.g., `date`, `number`, `varchar`).

Here's an example of a searched `case` expression:

```
CASE
  WHEN category.name IN ('Children','Family','Sports','Animation')
    THEN 'All Ages'
  WHEN category.name = 'Horror'
    THEN 'Adult'
  WHEN category.name IN ('Music','Games')
    THEN 'Teens'
  ELSE 'Other'
END
```

This `case` expression returns a string that can be used to classify films depending on their category. When the `case` expression is evaluated, the `when` clauses are evaluated in order from top to bottom; as soon as one of the conditions in a `when` clause evaluates to `true`, the corresponding expression is returned, and any remaining `when` clauses are ignored. If none of the `when` clause conditions evaluates to `true`, then the expression in the `else` clause is returned.

Although the previous example returns string expressions, keep in mind that `case` expressions may return any type of expression, including subqueries. Here's another version of the query from earlier in the chapter that uses a subquery to return the number of rentals, but only for active customers:

```
mysql> SELECT c.first_name, c.last_name,
    ->   CASE
    ->     WHEN active = 0 THEN 0
    ->     ELSE
    ->      (SELECT count(*) FROM rental r
    ->       WHERE r.customer_id = c.customer_id)
    ->   END num_rentals
    -> FROM customer c;
+------------+-------------+-------------+
| first_name  | last_name    | num_rentals |
+------------+-------------+-------------+
| MARY       | SMITH       |         32 |
| PATRICIA    | JOHNSON      |         27 |
| LINDA       | WILLIAMS     |         26 |
| BARBARA     | JONES        |         22 |
| ELIZABETH   | BROWN        |         38 |
| JENNIFER    | DAVIS        |         28 |
...
| TERRANCE    | ROUSH        |          0 |
| RENE        | MCALISTER    |         26 |
| EDUARDO     | HIATT        |         27 |
| TERRENCE    | GUNDERSON    |         30 |
| ENRIQUE     | FORSYTHE     |         28 |
| FREDDIE     | DUGGAN       |         25 |
| WADE        | DELVALLE     |         22 |
| AUSTIN      | CINTRON      |         19 |
+------------+-------------+-------------+
599 rows in set (0.01 sec)
```

This version of the query uses a correlated subquery to retrieve the number of rentals for each active customer. Depending on the percentage of active customers, using this approach may be more efficient than joining the `customer` and `rental` tables and grouping on the `customer_id` column.

Simple case Expressions

The *simple* `case` *expression* is quite similar to the searched `case` expression but is a bit less flexible. Here's the syntax:

```
CASE V0
  WHEN V1 THEN E1
  WHEN V2 THEN E2
  ...
```

```
  WHEN VN THEN EN
  [ELSE ED]
END
```

In the preceding definition, `V0` represents a value, and the symbols `V1`, `V2`, ..., `VN` represent values that are to be compared to `V0`. The symbols `E1`, `E2`, ..., `EN` represent expressions to be returned by the `case` expression, and `ED` represents the expression to be returned if none of the values in the set `V1`, `V2`, ..., `VN` matches the `V0` value.

Here's an example of a simple `case` expression:

```
CASE category.name
  WHEN 'Children' THEN 'All Ages'
  WHEN 'Family' THEN 'All Ages'
  WHEN 'Sports' THEN 'All Ages'
  WHEN 'Animation' THEN 'All Ages'
  WHEN 'Horror' THEN 'Adult'
  WHEN 'Music' THEN 'Teens'
  WHEN 'Games' THEN 'Teens'
  ELSE 'Other'
END
```

Simple `case` expressions are less flexible than searched `case` expressions because you can't specify your own conditions, whereas searched `case` expressions may include range conditions, inequality conditions, and multipart conditions using `and` / `or` / `not`, so I would recommend using searched `case` expressions for all but the simplest logic.

Examples of case Expressions

The following sections present a variety of examples illustrating the utility of conditional logic in SQL statements.

Result Set Transformations

You may have run into a situation where you are performing aggregations over a finite set of values, such as days of the week, but you want the result set to contain a single row with one column per value instead of one row per value. As an example, let's say you

have been asked to write a query that shows the number of film rentals for May, June, and July of 2005:

```
mysql> SELECT monthname(rental_date) rental_month,
    ->   count(*) num_rentals
    -> FROM rental
    -> WHERE rental_date BETWEEN '2005-05-01' AND '2005-08-01'
    -> GROUP BY monthname(rental_date);
+--------------+-------------+
| rental_month | num_rentals |
+--------------+-------------+
| May          |        1156 |
| June         |        2311 |
| July         |        6709 |
+--------------+-------------+
3 rows in set (0.01 sec)
```

However, you have also been instructed to return a single row of data with three columns (one for each of the three months). To transform this result set into a single row, you will need to create three columns and, within each column, sum *only* those rows pertaining to the month in question:

```
mysql> SELECT
    ->   SUM(CASE WHEN monthname(rental_date) = 'May' THEN 1
    ->       ELSE 0 END) May_rentals,
    ->   SUM(CASE WHEN monthname(rental_date) = 'June' THEN 1
    ->       ELSE 0 END) June_rentals,
    ->   SUM(CASE WHEN monthname(rental_date) = 'July' THEN 1
    ->       ELSE 0 END) July_rentals
    -> FROM rental
    -> WHERE rental_date BETWEEN '2005-05-01' AND '2005-08-01';
+-------------+--------------+--------------+
| May_rentals | June_rentals | July_rentals |
+-------------+--------------+--------------+
|        1156 |         2311 |         6709 |
+-------------+--------------+--------------+
1 row in set (0.01 sec)
```

Each of the three columns in the previous query are identical, except for the month value. When the `monthname()` function returns the desired value for that column, the `case` expression returns the value `1`; otherwise, it returns a `0`. When summed over all rows, each column returns the number of accounts opened for that month. Obviously,

such transformations are practical for only a small number of values; generating one column for each year since 1905 would quickly become tedious.

NOTE

Although it is a bit advanced for this book, it is worth pointing out that both SQL Server and Oracle Database include `pivot` clauses specifically for these types of queries.

Checking for Existence

Sometimes you will want to determine whether a relationship exists between two entities without regard for the quantity. For example, you might want to know whether an actor has appeared in at least one G-rated film, without regard for the actual number of films. Here's a query that uses multiple `case` expressions to generate three output columns, one to show whether the actor has appeared in G-rated films, another for PG-rated films, and a third for NC-17-rated films:

```
mysql> SELECT a.first_name, a.last_name,
    ->   CASE
    ->     WHEN EXISTS (SELECT 1 FROM film_actor fa
    ->                  INNER JOIN film f ON fa.film_id = f.film_id
    ->                WHERE fa.actor_id = a.actor_id
    ->                  AND f.rating = 'G') THEN 'Y'
    ->     ELSE 'N'
    ->   END g_actor,
    ->   CASE
    ->     WHEN EXISTS (SELECT 1 FROM film_actor fa
    ->                  INNER JOIN film f ON fa.film_id = f.film_id
    ->                WHERE fa.actor_id = a.actor_id
    ->                  AND f.rating = 'PG') THEN 'Y'
    ->     ELSE 'N'
    ->   END pg_actor,
    ->   CASE
    ->     WHEN EXISTS (SELECT 1 FROM film_actor fa
    ->                  INNER JOIN film f ON fa.film_id = f.film_id
    ->                WHERE fa.actor_id = a.actor_id
    ->                  AND f.rating = 'NC-17') THEN 'Y'
    ->     ELSE 'N'
    ->   END nc17_actor
    -> FROM actor a
    -> WHERE a.last_name LIKE 'S%' OR a.first_name LIKE 'S%';
+------------+-------------+---------+----------+------------+
```

```
| first_name | last_name   | g_actor | pg_actor | nc17_actor |
+------------+-------------+---------+----------+------------+
| JOE        | SWANK       | Y       | Y        | Y          |
| SANDRA     | KILMER      | Y       | Y        | Y          |
| CAMERON    | STREEP      | Y       | Y        | Y          |
| SANDRA     | PECK        | Y       | Y        | Y          |
| SISSY      | SOBIESKI    | Y       | Y        | N          |
| NICK       | STALLONE    | Y       | Y        | Y          |
| SEAN       | WILLIAMS    | Y       | Y        | Y          |
| GROUCHO    | SINATRA     | Y       | Y        | Y          |
| SCARLETT   | DAMON       | Y       | Y        | Y          |
| SPENCER    | PECK        | Y       | Y        | Y          |
| SEAN       | GUINESS     | Y       | Y        | Y          |
| SPENCER    | DEPP        | Y       | Y        | Y          |
| SUSAN      | DAVIS       | Y       | Y        | Y          |
| SIDNEY     | CROWE       | Y       | Y        | Y          |
| SYLVESTER  | DERN        | Y       | Y        | Y          |
| SUSAN      | DAVIS       | Y       | Y        | Y          |
| DAN        | STREEP      | Y       | Y        | Y          |
| SALMA      | NOLTE       | Y       | N        | Y          |
| SCARLETT   | BENING      | Y       | Y        | Y          |
| JEFF       | SILVERSTONE | Y       | Y        | Y          |
| JOHN       | SUVARI      | Y       | Y        | Y          |
| JAYNE      | SILVERSTONE | Y       | Y        | Y          |
+------------+-------------+---------+----------+------------+
22 rows in set (0.00 sec)
```

Each `case` expression includes a correlated subquery against the `film_actor` and `film` tables; one looks for films with a G rating, the second for films with a PG rating, and the third for films with a NC-17 rating. Since each `when` clause uses the `exists` operator, the conditions evaluate to `true` as long as the actor has appeared in at least one film with the proper rating.

In other cases, you may care how many rows are encountered, but only up to a point. For example, the next query uses a simple `case` expression to count the number of copies in inventory for each film and then returns either `'Out Of Stock'`, `'Scarce'`, `'Available'`, or `'Common'`:

```
mysql> SELECT f.title,
    ->   CASE (SELECT count(*) FROM inventory i
    ->       WHERE i.film_id = f.film_id)
    ->     WHEN 0 THEN 'Out Of Stock'
    ->     WHEN 1 THEN 'Scarce'
```

```
    ->     WHEN 2 THEN 'Scarce'
    ->     WHEN 3 THEN 'Available'
    ->     WHEN 4 THEN 'Available'
    ->     ELSE 'Common'
    ->   END film_availability
    -> FROM film f
    -> ;
+----------------------------+------------------+
| title                      | film_availability |
+----------------------------+------------------+
| ACADEMY DINOSAUR           | Common           |
| ACE GOLDFINGER             | Available        |
| ADAPTATION HOLES           | Available        |
| AFFAIR PREJUDICE           | Common           |
| AFRICAN EGG                | Available        |
| AGENT TRUMAN               | Common           |
| AIRPLANE SIERRA            | Common           |
| AIRPORT POLLOCK            | Available        |
| ALABAMA DEVIL              | Common           |
| ALADDIN CALENDAR           | Common           |
| ALAMO VIDEOTAPE            | Common           |
| ALASKA PHANTOM             | Common           |
| ALI FOREVER                | Available        |
| ALICE FANTASIA             | Out Of Stock     |
...
| YOUNG LANGUAGE             | Scarce           |
| YOUTH KICK                 | Scarce           |
| ZHIVAGO CORE               | Scarce           |
| ZOOLANDER FICTION          | Common           |
| ZORRO ARK                  | Common           |
+----------------------------+------------------+
1000 rows in set (0.01 sec)
```

For this query, I stopped counting after 5, since every other number greater than 5 will be given the `'Common'` label.

Division-by-Zero Errors

When performing calculations that include division, you should always take care to ensure that the denominators are never equal to zero. Whereas some database servers, such as Oracle Database, will throw an error when a zero denominator is encountered, MySQL simply sets the result of the calculation to `null`, as demonstrated by the following:

```
mysql> SELECT 100 / 0;
+---------+
| 100 / 0 |
+---------+
|    NULL |
+---------+
1 row in set (0.00 sec)
```

To safeguard your calculations from encountering errors or, even worse, from being mysteriously set to `null`, you should wrap all denominators in conditional logic, as demonstrated by the following:

```
mysql> SELECT c.first_name, c.last_name,
    ->   sum(p.amount) tot_payment_amt,
    ->   count(p.amount) num_payments,
    ->   sum(p.amount) /
    ->     CASE WHEN count(p.amount) = 0 THEN 1
    ->       ELSE count(p.amount)
    ->     END avg_payment
    -> FROM customer c
    ->   LEFT OUTER JOIN payment p
    ->   ON c.customer_id = p.customer_id
    -> GROUP BY c.first_name, c.last_name;
+------------+------------+-----------------+-------------+-------------+
| first_name | last_name  | tot_payment_amt | num_payments | avg_payment |
+------------+------------+-----------------+-------------+-------------+
| MARY       | SMITH      |          118.68 |          32 |    3.708750 |
| PATRICIA   | JOHNSON    |          128.73 |          27 |    4.767778 |
| LINDA      | WILLIAMS   |          135.74 |          26 |    5.220769 |
| BARBARA    | JONES      |           81.78 |          22 |    3.717273 |
| ELIZABETH  | BROWN      |          144.62 |          38 |    3.805789 |
...
| EDUARDO    | HIATT      |          130.73 |          27 |    4.841852 |
| TERRENCE   | GUNDERSON  |          117.70 |          30 |    3.923333 |
| ENRIQUE    | FORSYTHE   |           96.72 |          28 |    3.454286 |
| FREDDIE    | DUGGAN     |           99.75 |          25 |    3.990000 |
| WADE       | DELVALLE   |           83.78 |          22 |    3.808182 |
| AUSTIN     | CINTRON    |           83.81 |          19 |    4.411053 |
+------------+------------+-----------------+-------------+-------------+
599 rows in set (0.07 sec)
```

This query computes the average payment amount for each customer. Since some customers may be new and have yet to rent a film, it is best to include the `case` expression

to ensure that the denominator is never zero.

Conditional Updates

When updating rows in a table, you sometimes need conditional logic to generate a value for a column. For example, let's say that you run a job every week that will set the `customer.active` column to 0 for any customers who haven't rented a film in the last 90 days. Here's a statement that will set the value to either 0 or 1 for every customer:

```
UPDATE customer
SET active =
  CASE
    WHEN 90 <= (SELECT datediff(now(), max(rental_date))
                FROM rental r
                WHERE r.customer_id = customer.customer_id)
      THEN 0
    ELSE 1
  END
WHERE active = 1;
```

This statement uses a correlated subquery to determine the number of days since the last rental date for each customer and compares the value to 90; if the number returned by the subquery is 90 or higher, the customer is marked as inactive.

Handling Null Values

While `null` values are the appropriate thing to store in a table if the value for a column is unknown, it is not always appropriate to retrieve `null` values for display or to take part in expressions. For example, you might want to display the word *unknown* on a data entry screen rather than leaving a field blank. When retrieving the data, you can use a `case` expression to substitute the string if the value is `null`, as in:

```
SELECT c.first_name, c.last_name,
  CASE
    WHEN a.address IS NULL THEN 'Unknown'
    ELSE a.address
  END address,
  CASE
    WHEN ct.city IS NULL THEN 'Unknown'
    ELSE ct.city
  END city,
```

```
  CASE
    WHEN cn.country IS NULL THEN 'Unknown'
    ELSE cn.country
  END country
FROM customer c
  LEFT OUTER JOIN address a
  ON c.address_id = a.address_id
  LEFT OUTER JOIN city ct
  ON a.city_id = ct.city_id
  LEFT OUTER JOIN country cn
  ON ct.country_id = cn.country_id;
```

For calculations, `null` values often cause a `null` result, as demonstrated by the following example:

```
mysql> SELECT (7 * 5) / ((3 + 14) * null);
+----------------------------+
| (7 * 5) / ((3 + 14) * null) |
+----------------------------+
|                       NULL |
+----------------------------+
1 row in set (0.08 sec)
```

When performing calculations, `case` expressions are useful for translating a `null` value into a number (usually 0 or 1) that will allow the calculation to yield a non-`null` value.

Test Your Knowledge

Challenge your ability to work through conditional logic problems with the examples that follow. When you're done, compare your solutions with those in Appendix B.

Exercise 11-1

Rewrite the following query, which uses a simple `case` expression, so that the same results are achieved using a searched `case` expression. Try to use as few `when` clauses as possible.

```
SELECT name,
  CASE name
    WHEN 'English' THEN 'latin1'
```

```
    WHEN 'Italian' THEN 'latin1'
    WHEN 'French' THEN 'latin1'
    WHEN 'German' THEN 'latin1'
    WHEN 'Japanese' THEN 'utf8'
    WHEN 'Mandarin' THEN 'utf8'
    ELSE 'Unknown'
  END character_set
FROM language;
```

Exercise 11-2

Rewrite the following query so that the result set contains a single row with five columns (one for each rating). Name the five columns `G`, `PG`, `PG_13`, `R`, and `NC_17`.

```
mysql> SELECT rating, count(*)
    -> FROM film
    -> GROUP BY rating;
+--------+----------+
| rating | count(*) |
+--------+----------+
| PG     |      194 |
| G      |      178 |
| NC-17  |      210 |
| PG-13  |      223 |
| R      |      195 |
+--------+----------+
5 rows in set (0.00 sec)
```

Previous chapter
< 10. Joins Revisited

Next chapter
12. Transactions >